

Programovací úlohy ke státnicím (C#)

Informatika - Umělá inteligence, zaměření Strojové učení (UI-SU) | MFF UK

Implementační otázky společné informatiky přímo podle zadání zkoušek. Jazyk: C# (volba majitele).

Proč zvlášť a jak to číst

Ve společné informatice je skoro vždy jedna **implementační** otázka (návrh tříd nebo kód). To je nejčastější příčina *nuly* na společné části - definici si vybavíš, ale „napište program“ tě zaskočí. Tenhle sešit sbírá opakované programovací typy ze zkoušek 2019-2026 s úplným řešením v C#:

- **Objektový návrh** (nejčastější): výrazový strom, maticová knihovna, graf - rozhraní, polymorfismus, princip open/closed, rozšiřitelnost.
- **Reprezentace dat**: čtení strukturovaného binárního souboru.
- **Vlákna a synchronizace**: producent-konzument semafore, oprava race condition.
- **Nízká úroveň**: alokátor paměti, ovladač zařízení (PIO), emulátor procesoru.

Jak procvičovat: přečti zadání, napiš kód *na papír* (na zkoušce není počítač), pak porovnej. Cíl není odladěný program, ale **správná struktura** (rozhraní, klíčové metody, ošetření okrajových případů) - to nese body. Zelené *Jádro* je kostra, kterou musíš umět z hlavy.

Souvislost: tyto úlohy odpovídají „no-zero“ drilum z úrovní 1-2; tedy jsou pohromadě a s plnějším kódem.

Obsah

1	Objektový návrh	1
2	Reprezentace dat a soubory	4
3	Vlákna a synchronizace	5
4	Nízkoúrovňové úlohy	6

1 Objektový návrh

Úloha 1. **programování (C#)** Objektový návrh: výrazový strom (eval + symbolická derivace)

Navrhněte objektový model aritmetického výrazu nad konstantami, proměnnými a operacemi + a $\cdot$. Umožněte (a) vyhodnocení v daném prostředí (hodnoty proměnných), (b) symbolickou derivaci podle proměnné, (c) rozšíření o novou operaci bez zásahu do stávajícího kódu. (*Typ úlohy: 2019-jaro Q4, 2020-léto Q4 - výrazové stromy.*)

Jedno rozhraní IExpr, každý druh uzlu je samostatná třída (polymorfismus, princip open/closed):

```
public interface IExpr {
    double Eval(ReadOnlyDictionary<string, double> env);
    IExpr Derive(string x); // symbolická derivace podle x
}

public sealed class Const : IExpr {
    private readonly double v;
    public Const(double v) => this.v = v;
```

```

public double Eval(ReadOnlyDictionary<string,double> env) => v;
public IExpr Derive(string x) => new Const(0);
}

public sealed class Var : IExpr {
    private readonly string name;
    public Var(string name) => this.name = name;
    public double Eval(ReadOnlyDictionary<string,double> env) => env[name];
    public IExpr Derive(string x) => new Const(name == x ? 1 : 0);
}

public sealed class Add : IExpr {
    private readonly IExpr a, b;
    public Add(IExpr a, IExpr b) { this.a = a; this.b = b; }
    public double Eval(ReadOnlyDictionary<string,double> env) => a.Eval(env) + b.Eval(env);
    public IExpr Derive(string x) => new Add(a.Derive(x), b.Derive(x));
}

public sealed class Mul : IExpr {
    private readonly IExpr a, b;
    public Mul(IExpr a, IExpr b) { this.a = a; this.b = b; }
    public double Eval(ReadOnlyDictionary<string,double> env) => a.Eval(env) * b.Eval(env);
    // pravidlo o součinu: (a*b)' = a'*b + a*b'
    public IExpr Derive(string x) => new Add(new Mul(a.Derive(x), b), new Mul(a, b.Derive(x)));
}

```

Použití: výraz $x \cdot x + 3$ je `new Add(new Mul(new Var("x"), new Var("x")), new Const(3))`; `Eval` s `{["x"]=2}` dá 7, `Derive("x")` dá symbolicky $x \cdot 1 + 1 \cdot x (+0)$.

Rozšiřitelnost (c): novou operaci (např. Sin) přidáme jako novou třídu implementující `IExpr` - *beze změny* stávajících tříd (open/closed). Sdílení podvýrazu (DAG) je zdarma: tentýž objekt `IExpr` lze odkázat z více míst (uzly jsou neměnné).

Jedno rozhraní s operacemi (`Eval`, `Derive`), uzel = třída implementující ho; rekurze přes děti. Derivace součinu = pravidlo $(ab)' = a'b + ab'$. Rozšíření = nová třída, ne `switch`.

Pozor (častá chyba): Nepoužívej `switch` na typ uzlu - to porušuje open/closed a u nového uzlu zapomeneš větve. Polymorfismus přes rozhraní je správně. Uzly drž neměnné (immutable) kvůli sdílení.

Úloha 2. programování (C#) Knihovna pro práci s maticemi (rozhraní + hustá/řídká)

Navrhněte knihovnu pro matice: společné rozhraní, hustou implementaci a *řídou* (sparse) implementaci téhož rozhraní, a operaci násobení pracující nad rozhraním. (Typ úlohy: 2025-jaro Q2 - knihovna pro výpočty s maticemi.)

```

public interface IMatrix {
    int Rows { get; }
    int Cols { get; }
    double this[int r, int c] { get; } // indexer pro čtení prvku
}

public sealed class DenseMatrix : IMatrix {
    private readonly double[,] a;
    public DenseMatrix(double[,] a) => this.a = a;
}

```

```

    public int Rows => a.GetLength(0);
    public int Cols => a.GetLength(1);
    public double this[int r, int c] => a[r, c];
}

// řídka matice: uloží jen nenulové prvky, ale navenek je to tatáž abstrakce
public sealed class SparseMatrix : IMatrix {
    private readonly Dictionary<(int r, int c), double> nz = new();
    public int Rows { get; }
    public int Cols { get; }
    public SparseMatrix(int rows, int cols) { Rows = rows; Cols = cols; }
    public void Set(int r, int c, double v) {
        if (v != 0) nz[(r, c)] = v; else nz.Remove((r, c));
    }
    public double this[int r, int c] => nz.TryGetValue((r, c), out var v) ? v : 0.0;
}

// násobení pracuje jen přes rozhraní -> funguje pro libovolnou kombinaci
public static class MatrixOps {
    public static DenseMatrix Multiply(IMatrix x, IMatrix y) {
        if (x.Cols != y.Rows) throw new ArgumentException("nekompatibilní rozměry");
        var r = new double[x.Rows, y.Cols];
        for (int i = 0; i < x.Rows; i++)
            for (int j = 0; j < y.Cols; j++) {
                double s = 0;
                for (int k = 0; k < x.Cols; k++) s += x[i, k] * y[k, j];
                r[i, j] = s;
            }
        return new DenseMatrix(r);
    }
}

```

Proč to takhle: operace závisí jen na IMatrix, takže přidání nové reprezentace (pásová, jednotková, ...) nevyžaduje změnu Multiply (polymorfismus přes indexer). Tovární metoda (IMatrix Create(...)) by volajícím skryla výběr husté/řídke podle hustoty dat.

Rozhraní IMatrix (rozměry + indexer); hustá i řídka jsou jen různé implementace; algoritmy (násobení) píšes proti rozhraní, ne proti konkrétní třídě. Násobení $O(n^3)$, kontroluj rozměry.

Pozor (častá chyba): Násobení vyžaduje $x.Cols == y.Rows$. Řídka matice nesmí ukládat nuly (jinak ztrácí smysl); indexer vrací 0 pro chybějící klíč. Indexer je v C# `this[int,int]`.

Úloha 3. **programování (C#)** Objektový návrh grafu a značkování komponent

Navrhněte rozhraní grafu a implementaci seznamem sousedu. Napište značkování komponent souvislosti (každému vrcholu přiřadit id komponenty). (Typ úlohy: 2022-podzim Q4, 2019-podzim Q2 - programování + grafy.)

```

public interface IGraph {
    int N { get; }
    IEnumerable<int> Neighbors(int v);
}

public sealed class AdjListGraph : IGraph {
    private readonly List<int>[] adj;
    public AdjListGraph(int n) {
        adj = new List<int>[n];
    }
}

```

```

        for (int i = 0; i < n; i++) adj[i] = new List<int>();
    }
    public int N => adj.Length;
    public void AddEdge(int u, int v) { adj[u].Add(v); adj[v].Add(u); } // neorientovany
    public IEnumerable<int> Neighbors(int v) => adj[v];
}

// znakovani komponent pomoci BFS; vraci comp[v] = id komponenty
public static int[] Components(IGraph g) {
    var comp = new int[g.N];
    Array.Fill(comp, -1);
    int c = 0;
    for (int s = 0; s < g.N; s++) {
        if (comp[s] != -1) continue; // uz navstiveny
        var q = new Queue<int>();
        q.Enqueue(s); comp[s] = c;
        while (q.Count > 0) {
            int v = q.Dequeue();
            foreach (int w in g.Neighbors(v))
                if (comp[w] == -1) { comp[w] = c; q.Enqueue(w); }
        }
        c++; // dalsi komponenta
    }
    return comp;
}

```

Složitost $O(V + E)$ (každý vrchol/hrana zpracován jednou). Algoritmus závisí jen na IGraph, takže funguje i pro jinou reprezentaci (matice sousednosti) bez změny.

Implementace Komponent (C#)

Rozhraní IGraph (počet vrcholu + sousedé); reprezentace = seznam sousedu. Komponenty: pro každý nenavštívený vrchol spustí BFS/DFS a obarví dosažené stejným id, $O(V + E)$.

Pozor (častá chyba): Algoritmus piš proti IGraph, ne proti konkrétní třídě. Nezapomeň na vrcholy v různých komponentách (vnější smyčka přes všechny vrcholy). Pole comp inicializuj na -1 (nenavštíveno).

2 Reprezentace dat a soubory

Úloha 4. programování (C#) Čtení strukturovaného binárního souboru

Binární soubor: 4 bajty ASCII „REC1” (magic), uint32 count (little-endian), poté count záznamů, každý int32 id + float32 value (LE). Napište načtení do pole struktur a ošetřete poškozený soubor. (Typ úlohy: 2024-podzim Q2, 2025-podzim Q2, reprezentace dat / binární soubor.)

```

public readonly record struct Rec(int Id, float Value);

public static Rec[] Load(string path) {
    using var fs = File.OpenRead(path);
    using var br = new BinaryReader(fs); // cte little-endian

    var magic = br.ReadBytes(4); // ReadBytes muze vratit MENE bajtu
    if (magic.Length != 4 ||
        magic[0] != (byte)'R' || magic[1] != (byte)'E' ||

```

```

        magic[2] != (byte)'C' || magic[3] != (byte)'1')
        throw new InvalidDataException("spatny magic");

        uint count = br.ReadUInt32();
        // obrana proti nesmyslnemu count: over proti zbyvajici delce souboru
        long remaining = fs.Length - fs.Position;
        if ((long)count * 8 != remaining)
            throw new InvalidDataException("delka neodpovida count");

        var recs = new Rec[count];
        for (uint i = 0; i < count; i++) {
            int id = br.ReadInt32(); // pri zkraceni vyhodi EndOfStreamException
            float val = br.ReadSingle();
            recs[i] = new Rec(id, val);
        }
        return recs;
    }
}

```

Ruční složení `int32` z bajtu (princip endianity, nezávislé na CPU): `id = b0 | b1<<8 | b2<<16 | b3<<24`. Na big-endian CPU by selhalo až nativní přetypování (`BitConverter.ToInt32`), ne tohle explicitní složení; přenositelně lze použít `BinaryPrimitives.ReadInt32LittleEndian`.

`BinaryReader` čte LE; ověř `magic`, ověř `count` proti délce souboru před alokací, čti záznamy. `ReadBytes` vrací i méně bajtu (kontroluj délku), `ReadInt32` při zkrácení vyhodí výjimku.

Pozor (častá chyba): Nealokuj `new Rec[count]` bez ověření `count` (riziko OOM u poškozeného souboru). Little-endian = nejméně významný bajt první. Zarovnání ve *formátu* souboru nepředpokládej.

3 Vlákna a synchronizace

Úloha 5. **programování (C#)** Vlákna: producent-konzument semaforey a oprava race condition

(a) Napište producent-konzument s omezeným bufferem pomocí semaforu. (b) Ukažte časově závislou chybu (race condition) na sdíleném čítači a opravte ji. (*Typ úlohy: 2024-léto Q2, 2026-jaro Q2 - Semafor; 2023-podzim Q1 - race condition.*)

(a) Producent-konzument (buffer kapacity N): semafor `empty` počítá volná místa, `full` obsazená, lock chrání frontu.

```

var empty = new SemaphoreSlim(N, N); // volna mista
var full = new SemaphoreSlim(0, N); // obsazena mista
var gate = new object();
var buffer = new Queue<int>();

void Produce(int item) {
    empty.Wait(); // pockej na volne misto (P)
    lock (gate) { buffer.Enqueue(item); }
    full.Release(); // pribyl prvek (V)
}

int Consume() {
    full.Wait(); // pockej na prvek (P)
    int item; lock (gate) { item = buffer.Dequeue(); }
}

```

```

    empty.Release(); // uvolnil se slot (V)
    return item;
}

```

(b) **Race condition:** `counter++` není atomické (load, inkrement, store); dvě vlákna se mohou proložit a jeden inkrement se ztratí. Oprava zámkem nebo atomickou operací:

```

// chybné: for(...) counter++; z N vláken -> vysledek < N*kroku
// spravne A) zamek:
lock (gate) { counter++; }
// spravne B) atomicky:
Interlocked.Increment(ref counter);

```

Semafor: Wait=P (sniž, blokuj při 0), Release=V. Pořadí: *nejdřív* `empty.Wait()/full.Wait()`, *pak* lock. Race: nechráněný `counter++` ztrácí inkrementy; oprava lock nebo Interlocked.

Pozor (častá chyba): Nezaměň pořadí semaforu a zámku (čekej na zdroj PŘED zámkem, jinak deadlock). `empty` a `full` se nesmí prohodit. `counter++` bez synchronizace je klasická race.

4 Nízkoúrovňové úlohy

Úloha 6. programování (C#) Jednoduchý správce paměti (first-fit alokátor)

Implementujte alokátor nad blokem o dané velikosti: Alloc strategií first-fit (s rozdělením bloku) a Free (se slučováním sousedních volných bloku). (*Typ úlohy: 2025-léto Q2 - jednoduchý správce paměti.*)

```

public sealed class Allocator {
    public sealed class Block { // slouzi i jako "handle" pro Free
        public int Offset, Size;
        public bool Free = true;
        public Block? Next, Prev;
    }
    private readonly Block head;
    public Allocator(int size) => head = new Block { Offset = 0, Size = size };
    private static int Align(int n, int a) => (n + a - 1) / a * a;

    public Block? Alloc(int n) {
        n = Align(n, 8);
        for (var b = head; b != null; b = b.Next)
            if (b.Free && b.Size >= n) { // first-fit: první dost velký
                if (b.Size > n) Split(b, n);
                b.Free = false;
                return b;
            }
        return null; // nenalezeno
    }

    private void Split(Block b, int n) { // odstrihni zbytek jako nový volný blok
        var rest = new Block { Offset = b.Offset + n, Size = b.Size - n,
                                Next = b.Next, Prev = b };
        if (b.Next != null) b.Next.Prev = rest;
        b.Next = rest; b.Size = n;
    }
}

```

```

public void Free(Block b) {
    b.Free = true;
    if (b.Next is { Free: true } n) { // sluc s pravym susedem
        b.Size += n.Size; b.Next = n.Next;
        if (n.Next != null) n.Next.Prev = b;
    }
    if (b.Prev is { Free: true } p) { // sluc s levym susedem
        p.Size += b.Size; p.Next = b.Next;
        if (b.Next != null) b.Next.Prev = p;
    }
}
}

```

Fragmentace: *vnitřní* = zaokrouhlení na zarovnání nechá nevyužitý zbytek v bloku; *vnější* = volná paměť roztržštěná na malé kousky. Slučování ve **Free** brání vnější fragmentaci.

Index: Základní pojmy a algoritmy

Spojový seznam bloku (offset, velikost, volný?). Alloc = první dost velký volný blok, případně rozděl. Free = označ volný a slučuj s volnými sousedy. Zarovnání zaokrouhlí velikost nahoru.

Pozor (častá chyba): Po **Free** se MUSÍ slučovat sousední volné bloky, jinak vnější fragmentace roste. Hlavička/režie bloku zabírá místo. First-fit je rychlý, ale ne nejúspěšnější (vs best-fit).

Úloha 7. **programování (C#)** Ovladač zařízení programovaným V/V (PIO)

Pro řadič disku s porty STATUS, COMMAND a DATA napište programem řízenou obsluhu (PIO): vydejte příkaz čtení a přečtete 512 bajtu polling-em stavového registru. Vysvětlete variantu s přerušením. (*Typ úlohy: 2024-jaro Q2 - ovladač pro řadič disku; 2021-podzim - myš.*)

Řadič má registry adresované jako V/V porty; CPU s ním komunikuje instrukcemi in/out (zde abstraktně Inb/Outb).

```

const int DATA = 0x1F0, STATUS = 0x1F7, COMMAND = 0x1F7;
const int BSY = 0x80, DRQ = 0x08; // bity stavoveho registru

static byte Inb(int port) { /* HW: precti bajt z portu */ return 0; }
static void Outb(int port, byte v) { /* HW: zapis bajt na port */ }

static byte[] ReadSector() { // PIO ctene polling-em
    Outb(COMMAND, 0x20); // prikaz READ SECTOR
    var buf = new byte[512];
    for (int i = 0; i < 512; i++) {
        while ((Inb(STATUS) & BSY) != 0) { } // cekej, az radic neni zaneprazdnen
        while ((Inb(STATUS) & DRQ) == 0) { } // cekej, az jsou data pripravena
        buf[i] = Inb(DATA); // precti bajt dat
    }
    return buf;
}

```

Polling vs přerušení: polling *aktivně* cyklí na stavovém registru a pálí CPU. *Přerušení:* řadič po dokončení vyvolá IRQ; procesor přeruší běžící kód, skočí do obslužné rutiny (ISR), ta přečte data a oznámí dokončení (nastaví událost / probudí čekající vlákno). Procesor: dokončí instrukci, uloží kontext, podle vektoru přerušení skočí do ISR; OS: naplánuje obsluhu, po **iret** obnoví kontext.

Úloha 8. programování (C#) Emulátor jednoduchého procesoru (instrukční dispatch)

Registry řadiče = V/V porty (in/out). PIO polling: vydej příkaz, opakovaně čti STATUS (BSY/DRQ), pak čti DATA. Přerušeni místo pollingu: IRQ → ISR přečte data → probudí čekajícího (nepálí CPU).

Pozor (častá chyba): Polling plýtvá CPU - u pomalých zařízení se použije přerušeni. Pořadí bitu: čekej, až BSY=0 a DRQ=1, teprve pak čti DATA. Příkaz se zapisuje do COMMAND, data se čtou z DATA.

Úloha 8. programování (C#) Emulátor jednoduchého procesoru (instrukční dispatch)

Napište emulátor jednoduchého registrového procesoru s instrukcemi LOAD, ADD, SUB, JMP, JZ, HALT. Program je pole instrukcí, stav jsou registry a čítač instrukcí (PC). (*Typ úlohy: 2022-léto Q4 - emulátor procesoru.*)

```
public enum Op { Load, Add, Sub, Jmp, Jz, Halt }
public readonly record struct Instr(Op Op, int A, int B);

public static int[] Run(Instr[] prog, int regCount) {
    var reg = new int[regCount];
    int pc = 0;
    while (true) {
        var ins = prog[pc];
        switch (ins.Op) {
            case Op.Load: reg[ins.A] = ins.B; pc++; break; // reg[A] = B
            case Op.Add: reg[ins.A] += reg[ins.B]; pc++; break; // reg[A]+=reg[B]
            case Op.Sub: reg[ins.A] -= reg[ins.B]; pc++; break;
            case Op.Jmp: pc = ins.A; break; // skok na adresu A
            case Op.Jz: pc = (reg[ins.A] == 0) ? ins.B : pc + 1; break; // if reg[A]==0 goto B
            case Op.Halt: return reg;
            default: throw new InvalidOperationException("neznama instrukce");
        }
    }
}
```

Princip: smyčka *fetch-decode-execute* - načti instrukci na pc, rozskoč podle opkódu (switch), proved, aktualizuj pc (skoky mění pc přímo). *Alternativa OO:* každá instrukce jako třída implementující `IInstr.Execute(state)` - dispatch přes polymorfismus místo switch (snáz rozšiřitelné).

Úloha 8. programování (C#) Emulátor jednoduchého procesoru (instrukční dispatch)

Stav = registry + PC. Smyčka *fetch-decode-execute*: switch podle opkódu provede instrukci a posune PC; skoky nastaví PC přímo; HALT ukončí. OO varianta = instrukce jako třídy s `Execute`.

Pozor (častá chyba): Po skoku PC *neinkrementuj* (skok ho už nastavil). JZ testuje registr a větví. Bez HALT / chybného PC hrozí nekonečná smyčka nebo přístup mimo pole - ošetři.